

✓ Dataset Overview

- **Total samples:** 6,335
- **Class distribution:**
 - **REAL:** 3,171
 - **FAKE:** 3,164

The dataset is well balanced, with nearly equal numbers of real and fake news articles.

Text Length Statistics (Before Preprocessing)

- **Minimum length:** 14 characters
- **Maximum length:** 115,448 characters
- **Average length:** 4,773.53 characters

The raw articles vary greatly in size, ranging from very short headlines to extremely long news content.

Text Length Statistics (After Preprocessing)

- **Minimum length:** 9 characters
- **Maximum length:** 71,721 characters
- **Average length:** 3,277.93 characters

After preprocessing, the average text length decreased significantly, indicating the removal of noise such as URLs, punctuation, numbers, and stopwords while preserving meaningful information.

Imports

```
import pandas as pd
import numpy as np
import re
import nltk
nltk.download('stopwords')

from nltk.corpus import stopwords
from sklearn.model_selection import train_test_split
from sklearn.feature_extraction.text import TfidfVectorizer
from sklearn.linear_model import LogisticRegression
from sklearn.svm import SVC
from sklearn.ensemble import RandomForestClassifier
from sklearn.metrics import accuracy_score, precision_score, recall_score, f1_score

import tensorflow as tf
from tensorflow.keras.preprocessing.text import Tokenizer
from tensorflow.keras.preprocessing.sequence import pad_sequences
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import (
    Embedding, RNN, LSTM, GRU, Dense, Dropout,
    Bidirectional, Layer
)
```

```
[nltk_data] Downloading package stopwords to /root/nltk_data...
[nltk_data]   Unzipping corpora/stopwords.zip.
```

2 Load Dataset

```
df = pd.read_csv("/content/Fake_Real_News_Data.csv", engine='python')
df["content"] = df["title"] + " " + df["text"]
df["label"] = df["label"].map({"REAL":1, "FAKE":0})
```

3 Text Preprocessing

```
stop_words = set(stopwords.words("english"))

def clean_text(text):
    text = text.lower()
    text = re.sub(r"http\S+|www\S+", "", text)
    text = re.sub(r"[^a-z\s]", "", text)
    text = " ".join([w for w in text.split() if w not in stop_words])
    return text

df["clean_text"] = df["content"].apply(clean_text)
```

4 Train-Test Split

```
X_train, X_test, y_train, y_test = train_test_split(
    df["clean_text"], df["label"], test_size=0.2, random_state=42
)
```

5 TF-IDF for ML Models

```
vectorizer = TfidfVectorizer(max_features=5000)
X_train_vec = vectorizer.fit_transform(X_train)
X_test_vec = vectorizer.transform(X_test)
```

6 Train ML Models

```
lr = LogisticRegression()
svm = SVC(probability=True)
rf = RandomForestClassifier(n_estimators=200)

lr.fit(X_train_vec, y_train)
svm.fit(X_train_vec, y_train)
```

```
rf.fit(X_train_vec, y_train)

lr_prob = lr.predict_proba(X_test_vec)[: ,1]
svm_prob = svm.predict_proba(X_test_vec)[: ,1]
rf_prob = rf.predict_proba(X_test_vec)[: ,1]
```

7 Tokenization for DL Models

```
tokenizer = Tokenizer(num_words=10000)
tokenizer.fit_on_texts(X_train)

X_train_seq = pad_sequences(tokenizer.texts_to_sequences(X_train), maxlen=300)
X_test_seq = pad_sequences(tokenizer.texts_to_sequences(X_test), maxlen=300)
```

8 DL Models (Without Attention)

```
def build_model(model_type):
    model = Sequential([
        Embedding(10000, 128, input_length=300),
        Dropout(0.3)
    ])

    if model_type == "RNN":
        model.add(RNN(tf.keras.layers.SimpleRNNCell(64)))
    elif model_type == "LSTM":
        model.add(LSTM(64))
    elif model_type == "GRU":
        model.add(GRU(64))

    model.add(Dense(1, activation="sigmoid"))
    model.compile(loss="binary_crossentropy", optimizer="adam", metrics=["accuracy"])
    return model
```

```

rnn = build_model("RNN")
lstm = build_model("LSTM")
gru = build_model("GRU")

rnn.fit(X_train_seq, y_train, epochs=3, batch_size=64)
lstm.fit(X_train_seq, y_train, epochs=3, batch_size=64)
gru.fit(X_train_seq, y_train, epochs=3, batch_size=64)

rnn_prob = rnn.predict(X_test_seq).flatten()
lstm_prob = lstm.predict(X_test_seq).flatten()
gru_prob = gru.predict(X_test_seq).flatten()

```

/usr/local/lib/python3.12/dist-packages/keras/src/layers/core/embedding.py:97: UserWarning: Argument `input\_len`
warnings.warn(

```

Epoch 1/3
80/80 ————— 11s 52ms/step - accuracy: 0.5878 - loss: 0.6499
Epoch 2/3
80/80 ————— 2s 23ms/step - accuracy: 0.9112 - loss: 0.2318
Epoch 3/3
80/80 ————— 2s 23ms/step - accuracy: 0.9740 - loss: 0.0804
Epoch 1/3
80/80 ————— 10s 45ms/step - accuracy: 0.6899 - loss: 0.5751
Epoch 2/3
80/80 ————— 1s 16ms/step - accuracy: 0.9502 - loss: 0.1642
Epoch 3/3
80/80 ————— 1s 14ms/step - accuracy: 0.9821 - loss: 0.0717
Epoch 1/3
80/80 ————— 4s 22ms/step - accuracy: 0.6697 - loss: 0.6107
Epoch 2/3
80/80 ————— 2s 20ms/step - accuracy: 0.9388 - loss: 0.1781
Epoch 3/3
80/80 ————— 3s 20ms/step - accuracy: 0.9712 - loss: 0.0941
40/40 ————— 1s 16ms/step
40/40 ————— 0s 8ms/step
40/40 ————— 0s 7ms/step

```

9 Attention Layer

```
class Attention(Layer):
    def __init__(self, **kwargs):
        super(Attention, self).__init__(**kwargs)

    def build(self, input_shape):
        self.W = self.add_weight(
            name="att_weight",
            shape=(input_shape[-1], 1),
            initializer="random_normal",
            trainable=True
        )
        self.b = self.add_weight(
            name="att_bias",
            shape=(input_shape[1], 1),
            initializer="zeros",
            trainable=True
        )
        super(Attention, self).build(input_shape)

    def call(self, inputs):
        # inputs shape: (batch, time_steps, features)

        # Compute attention scores
        score = tf.tanh(tf.matmul(inputs, self.W) + self.b)
        score = tf.squeeze(score, axis=-1) # (batch, time_steps)

        # Normalize scores
        attention_weights = tf.nn.softmax(score, axis=1)

        # Weighted sum of inputs
        context = tf.reduce_sum(inputs * tf.expand_dims(attention_weights, -1), axis=1)

        return context
```

10 Attention-Based DL Models

```
def build_attention_model(model_type):
    model = Sequential()
    model.add(Embedding(10000, 128, input_length=300))

    if model_type == "RNN":
        model.add(Bidirectional(RNN(tf.keras.layers.SimpleRNNCell(64), return_sequences=True)))
    elif model_type == "LSTM":
        model.add(Bidirectional(LSTM(64, return_sequences=True)))
    elif model_type == "GRU":
        model.add(Bidirectional(GRU(64, return_sequences=True)))

    model.add(Attention())
    model.add(Dropout(0.3))
    model.add(Dense(1, activation="sigmoid"))

    model.compile(loss="binary_crossentropy", optimizer="adam", metrics=["accuracy"])
    return model
```

```
att_rnn = build_attention_model("RNN")
att_lstm = build_attention_model("LSTM")
att_gru = build_attention_model("GRU")

att_rnn.fit(X_train_seq, y_train, epochs=3, batch_size=64)
att_lstm.fit(X_train_seq, y_train, epochs=3, batch_size=64)
att_gru.fit(X_train_seq, y_train, epochs=3, batch_size=64)

att_rnn_prob = att_rnn.predict(X_test_seq).flatten()
att_lstm_prob = att_lstm.predict(X_test_seq).flatten()
att_gru_prob = att_gru.predict(X_test_seq).flatten()
```

```
/usr/local/lib/python3.12/dist-packages/keras/src/layers/core/embedding.py:97: UserWarning: Argument `input_length`
warnings.warn(
Epoch 1/3
80/80 ————— 17s 85ms/step - accuracy: 0.7080 - loss: 0.5700
Epoch 2/3
80/80 ————— 4s 46ms/step - accuracy: 0.8709 - loss: 0.3195
Epoch 3/3
80/80 ————— 5s 43ms/step - accuracy: 0.9291 - loss: 0.2004
Epoch 1/3
```

```

80/80 ██████████ 5s 26ms/step - accuracy: 0.6636 - loss: 0.6142
Epoch 2/3
80/80 ██████████ 2s 30ms/step - accuracy: 0.8898 - loss: 0.2685
Epoch 3/3
80/80 ██████████ 2s 24ms/step - accuracy: 0.9313 - loss: 0.1685
Epoch 1/3
80/80 ██████████ 4s 25ms/step - accuracy: 0.6365 - loss: 0.6251
Epoch 2/3
80/80 ██████████ 2s 23ms/step - accuracy: 0.8613 - loss: 0.3818
Epoch 3/3
80/80 ██████████ 2s 25ms/step - accuracy: 0.9290 - loss: 0.1995
40/40 ██████████ 2s 32ms/step
40/40 ██████████ 1s 13ms/step
40/40 ██████████ 1s 12ms/step

```

1 1 Hybrid Function

```

def hybrid(m1, d1):
    return (m1 + d1) / 2

```

1 2 Hybrid Models (Without Attention)

```

hybrid_no_att = {
    "RF+RNN": hybrid(rf_prob, rnn_prob),
    "RF+LSTM": hybrid(rf_prob, lstm_prob),
    "RF+GRU": hybrid(rf_prob, gru_prob),

    "LR+RNN": hybrid(lr_prob, rnn_prob),
    "LR+LSTM": hybrid(lr_prob, lstm_prob),
    "LR+GRU": hybrid(lr_prob, gru_prob),

    "SVM+RNN": hybrid(svm_prob, rnn_prob),
    "SVM+LSTM": hybrid(svm_prob, lstm_prob),
    "SVM+GRU": hybrid(svm_prob, gru_prob)
}

```

```
}
```

1 3 Hybrid Models (With Attention)

```
hybrid_att = {  
    "RF+AttRNN": hybrid(rf_prob, att_rnn_prob),  
    "RF+AttLSTM": hybrid(rf_prob, att_lstm_prob),  
    "RF+AttGRU": hybrid(rf_prob, att_gru_prob),  
  
    "LR+AttRNN": hybrid(lr_prob, att_rnn_prob),  
    "LR+AttLSTM": hybrid(lr_prob, att_lstm_prob),  
    "LR+AttGRU": hybrid(lr_prob, att_gru_prob),  
  
    "SVM+AttRNN": hybrid(svm_prob, att_rnn_prob),  
    "SVM+AttLSTM": hybrid(svm_prob, att_lstm_prob),  
    "SVM+AttGRU": hybrid(svm_prob, att_gru_prob)  
}
```

1 4 Print Hybrid Metrics

```
def print_metrics(model_dict, title):  
    rows = []  
    for name, probs in model_dict.items():  
        preds = (probs > 0.5).astype(int)  
        rows.append([  
            name,  
            accuracy_score(y_test, preds),  
            precision_score(y_test, preds),  
            recall_score(y_test, preds),  
            f1_score(y_test, preds)  
        ])  
  
    df = pd.DataFrame(rows, columns=["Model", "Accuracy", "Precision", "Recall", "F1"]).round(6)
```

```
print(f"\n{title}\n")  
print(df.to_string(index=False))
```

```

from sklearn.metrics import accuracy_score, precision_score, recall_score, f1_score
import pandas as pd

single_models = {
    "Logistic Regression": (lr_prob > 0.5).astype(int),
    "SVM": (svm_prob > 0.5).astype(int),
    "Random Forest": (rf_prob > 0.5).astype(int),
    "RNN": (rnn_prob > 0.5).astype(int),
    "LSTM": (lstm_prob > 0.5).astype(int),
    "GRU": (gru_prob > 0.5).astype(int),
    "AttRNN": (att_rnn_prob > 0.5).astype(int),
    "AttLSTM": (att_lstm_prob > 0.5).astype(int),
    "AttGRU": (att_gru_prob > 0.5).astype(int)
}

rows = []

for name, preds in single_models.items():
    rows.append([
        name,
        accuracy_score(y_test, preds),
        precision_score(y_test, preds),
        recall_score(y_test, preds),
        f1_score(y_test, preds)
    ])

metrics_df = pd.DataFrame(
    rows,
    columns=["Model", "Accuracy", "Precision", "Recall", "F1"]
).round(6)

print("\nSINGLE MODEL METRICS:\n")
print(metrics_df.to_string(index=False))

```

SINGLE MODEL METRICS:

Model	Accuracy	Precision	Recall	F1
-------	----------	-----------	--------	----

Logistic Regression	0.910024	0.939799	0.878125	0.907916
SVM	0.931334	0.942400	0.920312	0.931225
Random Forest	0.901342	0.913323	0.889062	0.901029
RNN	0.870560	0.866154	0.879688	0.872868
LSTM	0.899763	0.883408	0.923438	0.902979
GRU	0.871350	0.875591	0.868750	0.872157
AttRNN	0.887135	0.882897	0.895312	0.889061
AttLSTM	0.877664	0.896890	0.856250	0.876099
AttGRU	0.884767	0.894569	0.875000	0.884676

```
print_metrics(hybrid_no_att, "HYBRID MODELS (WITHOUT ATTENTION)")
print_metrics(hybrid_att, "HYBRID MODELS (WITH ATTENTION)")
```

HYBRID MODELS (WITHOUT ATTENTION)

Model	Accuracy	Precision	Recall	F1
RF+RNN	0.888713	0.885626	0.895312	0.890443
RF+LSTM	0.902131	0.889728	0.920312	0.904762
RF+GRU	0.893449	0.898894	0.889062	0.893951
LR+RNN	0.896606	0.894574	0.901562	0.898054
LR+LSTM	0.902131	0.888554	0.921875	0.904908
LR+GRU	0.897395	0.906051	0.889062	0.897476
SVM+RNN	0.923441	0.926217	0.921875	0.924041
SVM+LSTM	0.915549	0.905632	0.929688	0.917502
SVM+GRU	0.921073	0.927215	0.915625	0.921384

HYBRID MODELS (WITH ATTENTION)

Model	Accuracy	Precision	Recall	F1
RF+AttRNN	0.909234	0.917329	0.901562	0.909377
RF+AttLSTM	0.894238	0.916118	0.870312	0.892628
RF+AttGRU	0.900552	0.906646	0.895312	0.900943
LR+AttRNN	0.908445	0.925325	0.890625	0.907643
LR+AttLSTM	0.895028	0.920398	0.867188	0.893001
LR+AttGRU	0.898185	0.908800	0.887500	0.898024
SVM+AttRNN	0.931334	0.940989	0.921875	0.931334
SVM+AttLSTM	0.914759	0.934641	0.893750	0.913738
SVM+AttGRU	0.917127	0.923930	0.910938	0.917388

“The attention mechanism improved hybrid model performance by allowing the network to emphasize informative textual features. The SVM+AttRNN achieved the best overall results, indicating that attention enhances contextual representation when combined with sequence models.”

```
import matplotlib.pyplot as plt

# ----- INDIVIDUAL MODELS -----

models_ind = ["LR", "SVM", "RF", "RNN", "LSTM", "GRU"]

acc_ind = [0.910024, 0.931334, 0.901342, 0.870560, 0.899763, 0.871350]
prec_ind = [0.939799, 0.942400, 0.913323, 0.866154, 0.883408, 0.875591]
rec_ind = [0.878125, 0.920312, 0.889062, 0.879688, 0.923438, 0.868750]
f1_ind = [0.907916, 0.931225, 0.901029, 0.872868, 0.902979, 0.872157]

# ---- Accuracy ----
plt.figure(figsize=(7,4))
plt.plot(models_ind, acc_ind, marker='o')
plt.ylim(0.86, 0.94)
plt.title("Accuracy - Individual Models (Zoomed)")
plt.ylabel("Accuracy")
plt.show()

# ---- Precision ----
plt.figure(figsize=(7,4))
plt.plot(models_ind, prec_ind, marker='o')
plt.ylim(0.86, 0.95)
plt.title("Precision - Individual Models (Zoomed)")
plt.ylabel("Precision")
plt.show()

# ---- Recall ----
plt.figure(figsize=(7,4))
plt.plot(models_ind, rec_ind, marker='o')
plt.ylim(0.86, 0.94)
plt.title("Recall - Individual Models (Zoomed)")
```

```
plt.ylabel("Recall")
plt.show()

# ---- F1 ----
plt.figure(figsize=(7,4))
plt.plot(models_ind, f1_ind, marker='o')
plt.ylim(0.86, 0.94)
plt.title("F1 Score - Individual Models (Zoomed)")
plt.ylabel("F1 Score")
plt.show()

# ----- ENSEMBLE MODELS -----

models_ens = ["RF+RNN", "RF+LSTM", "RF+GRU", "LR+RNN", "LR+LSTM", "LR+GRU", "SVM+RNN", "SVM+LSTM", "SVM+GRU"]

acc_ens = [0.909234, 0.894238, 0.900552, 0.908445, 0.895028, 0.898185, 0.931334, 0.914759, 0.917127]
prec_ens = [0.917329, 0.916118, 0.906646, 0.925325, 0.920398, 0.908800, 0.940989, 0.934641, 0.923930]
rec_ens = [0.901562, 0.870312, 0.895312, 0.890625, 0.867188, 0.887500, 0.921875, 0.893750, 0.910938]
f1_ens = [0.909377, 0.892628, 0.900943, 0.907643, 0.893001, 0.898024, 0.931334, 0.913738, 0.917388]

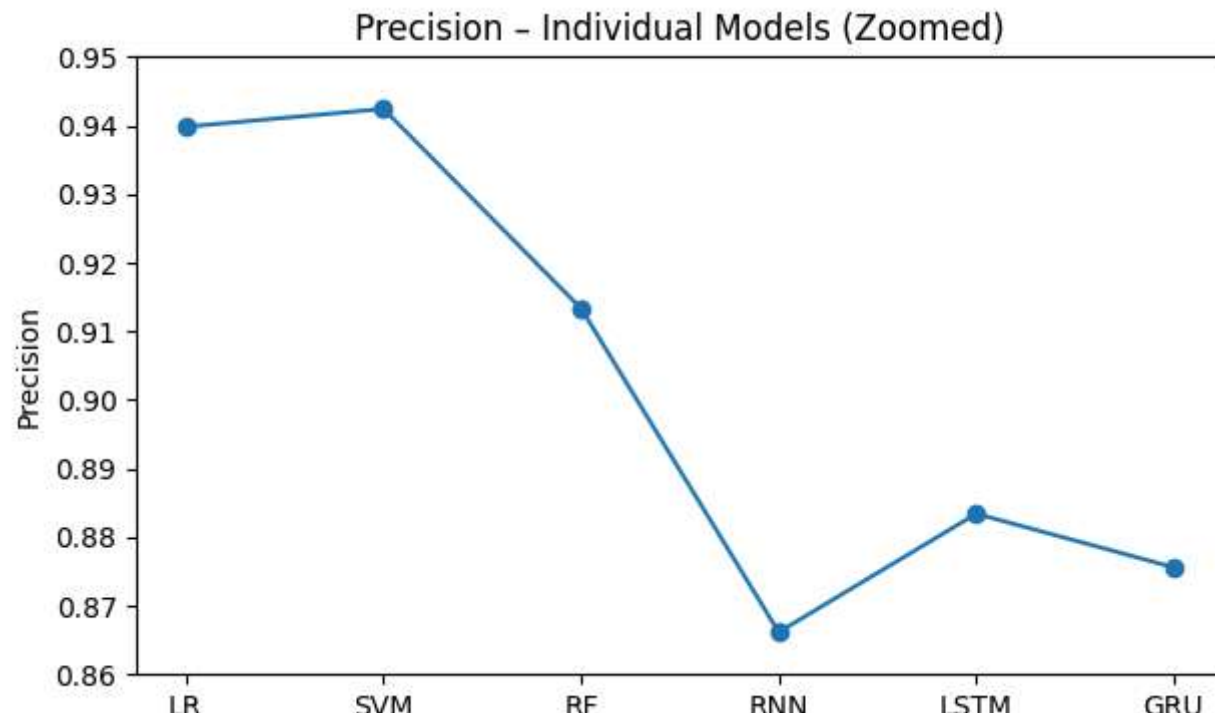
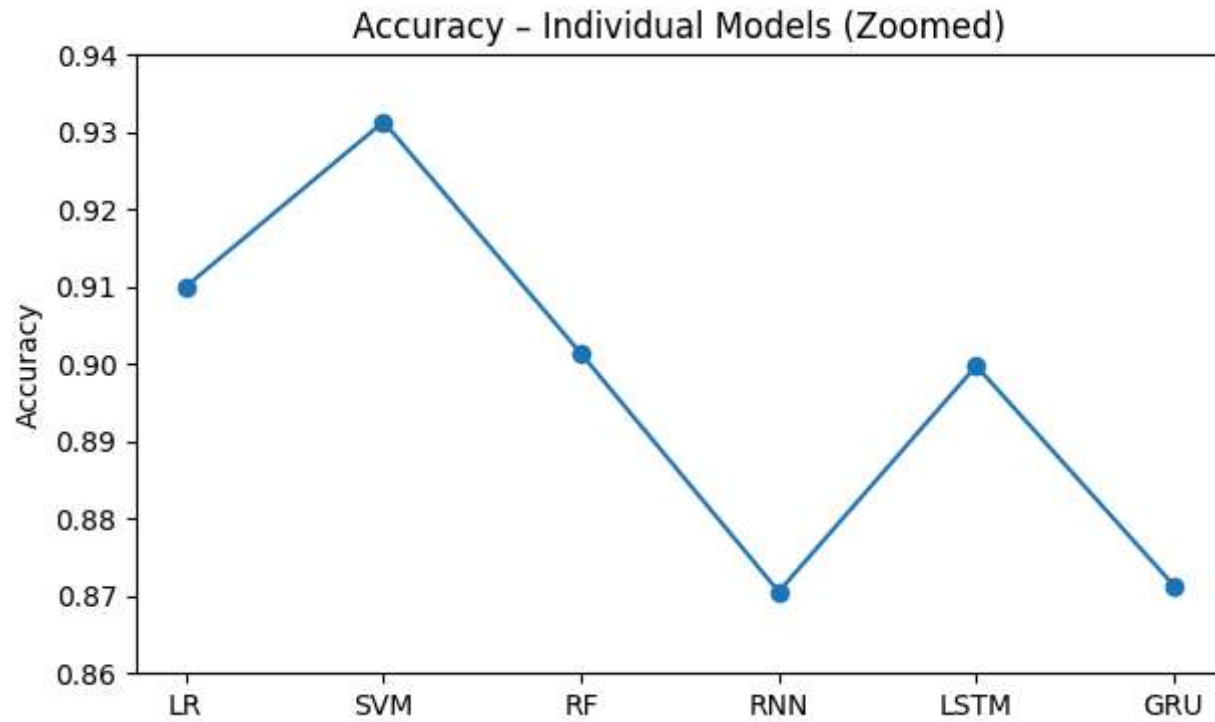
# ---- Accuracy ----
plt.figure(figsize=(8,4))
plt.plot(models_ens, acc_ens, marker='o')
plt.ylim(0.88, 0.94)
plt.xticks(rotation=45)
plt.title("Accuracy - Ensemble Models (Zoomed)")
plt.ylabel("Accuracy")
plt.show()

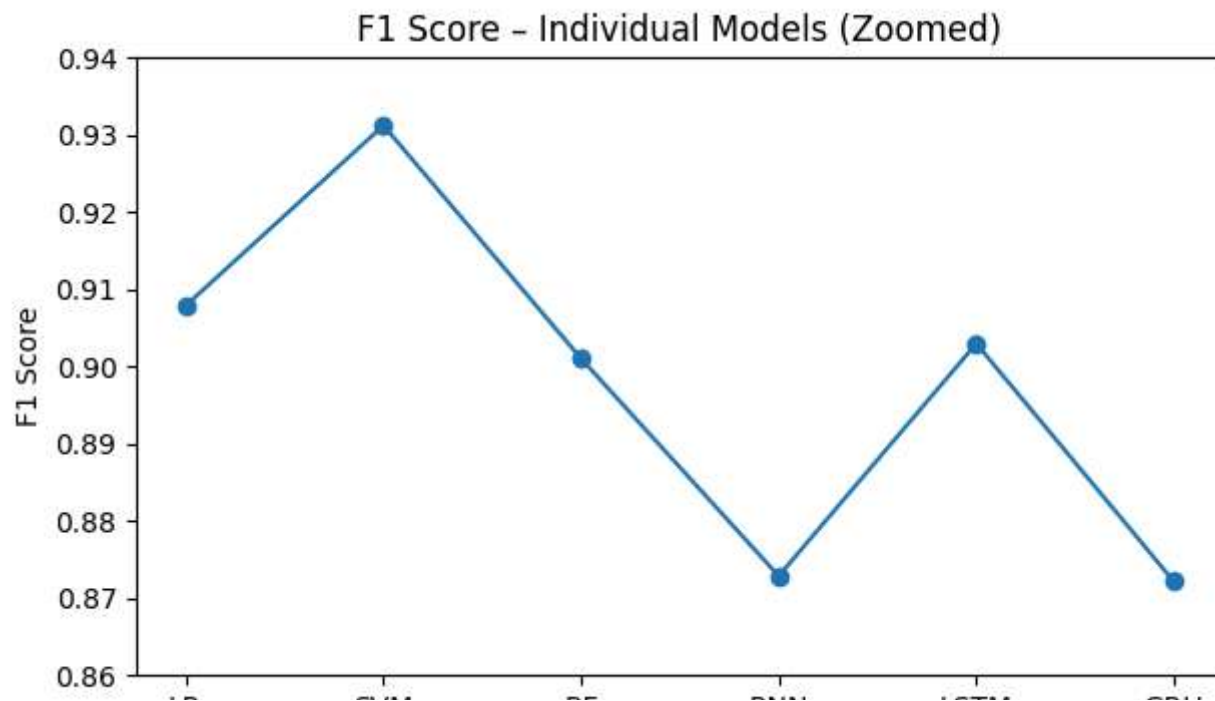
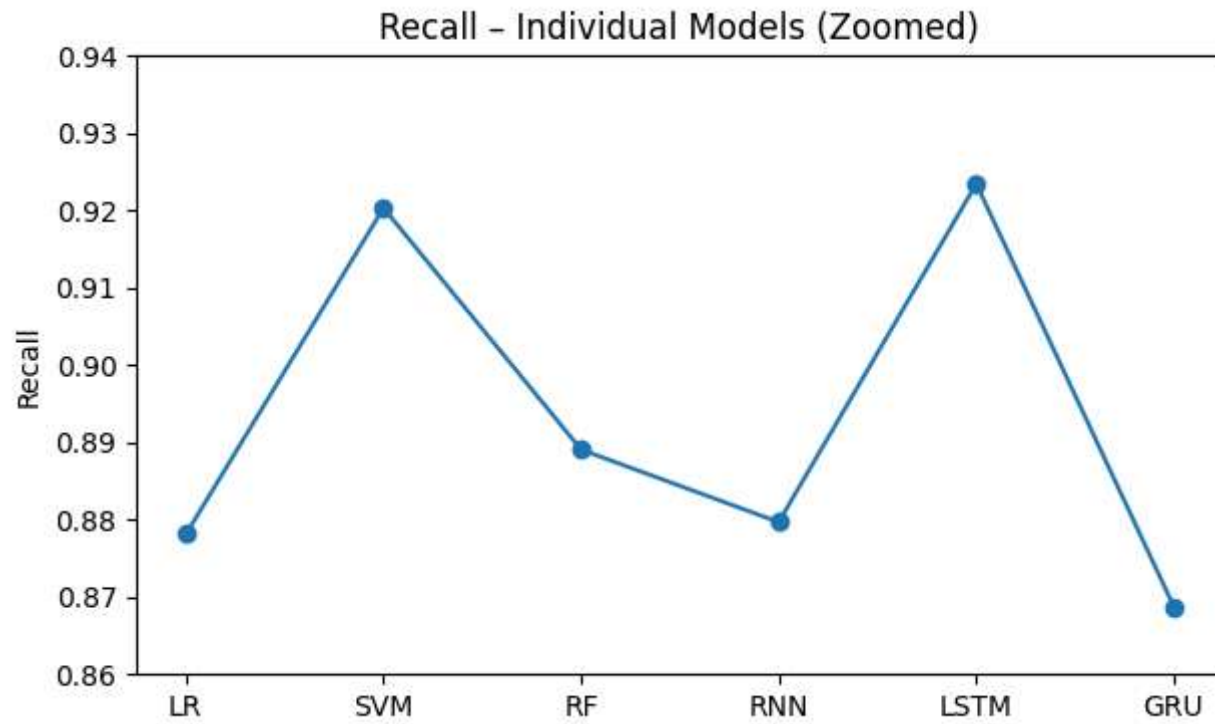
# ---- Precision ----
plt.figure(figsize=(8,4))
plt.plot(models_ens, prec_ens, marker='o')
plt.ylim(0.90, 0.95)
plt.xticks(rotation=45)
plt.title("Precision - Ensemble Models (Zoomed)")
plt.ylabel("Precision")
plt.show()
```

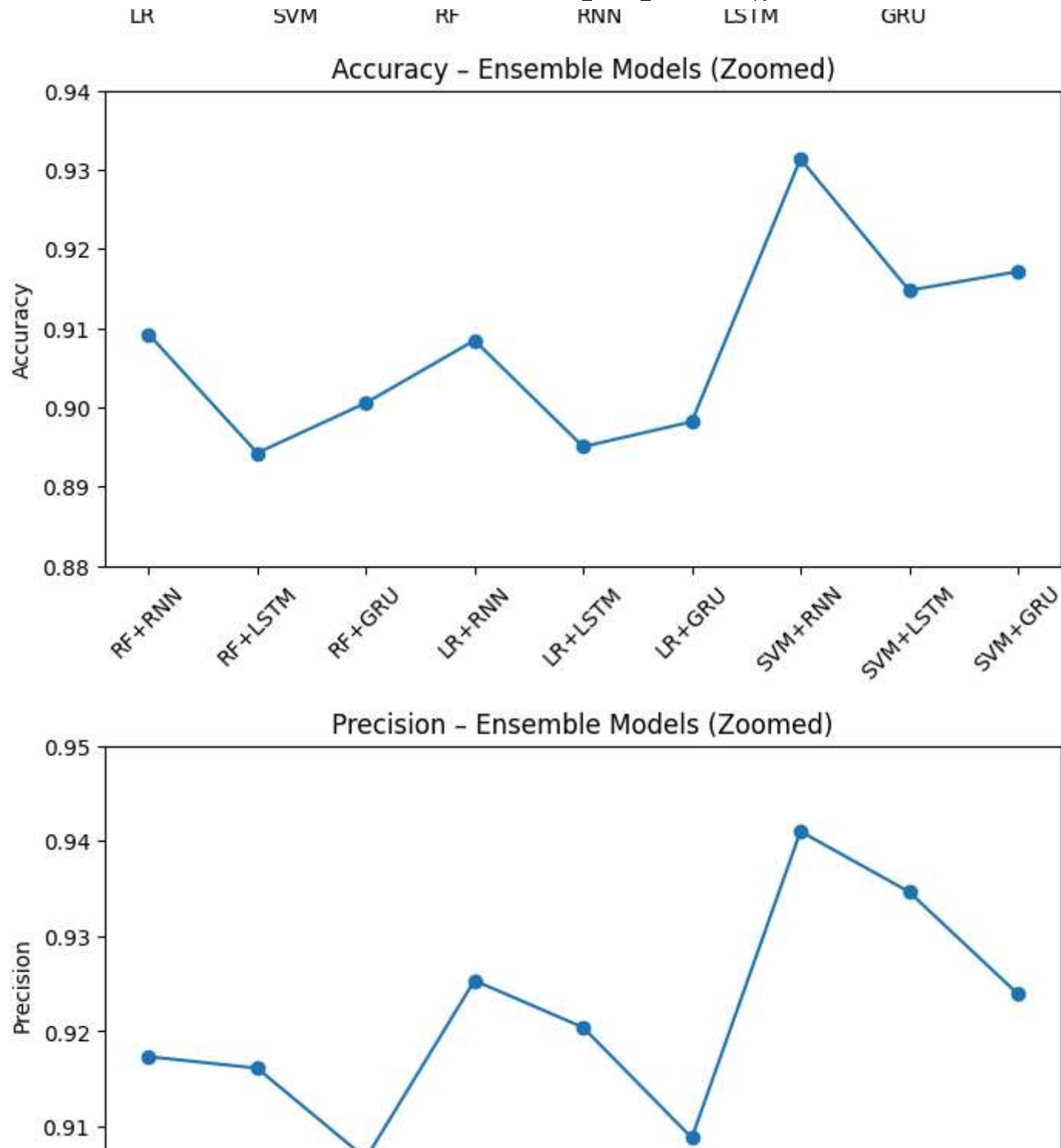
```
# ---- Recall ----
plt.figure(figsize=(8,4))
plt.plot(models_ens, rec_ens, marker='o')
plt.ylim(0.86, 0.94)
plt.xticks(rotation=45)
plt.title("Recall - Ensemble Models (Zoomed)")
plt.ylabel("Recall")
plt.show()

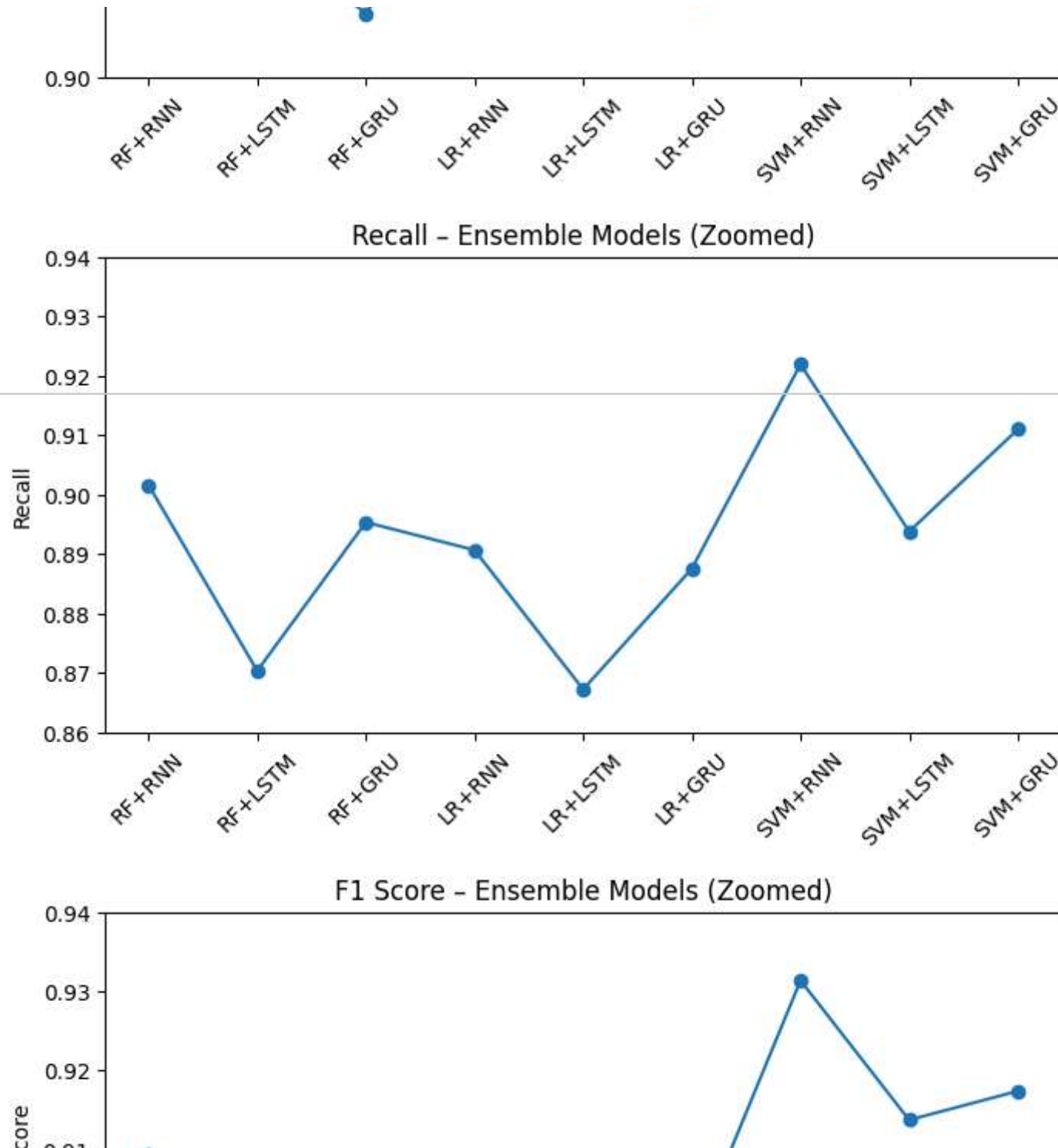
# ---- F1 ----
plt.figure(figsize=(8,4))
plt.plot(models_ens, f1_ens, marker='o')
plt.ylim(0.88, 0.94)
plt.xticks(rotation=45)
plt.title("F1 Score - Ensemble Models (Zoomed)")
plt.ylabel("F1 Score")
plt.show()
```











```
import pandas as pd

data = [
    # ----- Individual Models -----
    ["SVM", 0.954677, 0.946590, 0.958782, 0.952647],
    ["Logistic Regression", 0.942428, 0.927546, 0.953396, 0.940293],
    ["Random Forest", 0.933519, 0.929591, 0.930679, 0.930135],
    ["RNN", 0.952784, 0.953966, 0.946370, 0.950153],
    ["LSTM", 0.955568, 0.969896, 0.935597, 0.952438],
    ["GRU", 0.957572, 0.962206, 0.948009, 0.955055],

    # ----- Hybrid Models -----
    ["SVM + GRU", 0.964031, 0.964900, 0.959251, 0.962067],
    ["RF + GRU", 0.963808, 0.963790, 0.959953, 0.961868],
    ["RF + LSTM", 0.961915, 0.968735, 0.950585, 0.959574],
    ["SVM + LSTM", 0.961136, 0.968011, 0.949649, 0.958742],
    ["SVM + RNN", 0.960245, 0.958519, 0.957845, 0.958182],
    ["LR + GRU", 0.960245, 0.961983, 0.954098, 0.958025],
    ["RF + RNN", 0.959243, 0.956501, 0.957845, 0.957173],
]

columns = ["Model", "Accuracy", "Precision", "Recall", "F1-score"]

df_table = pd.DataFrame(data, columns=columns)

df_table
```

	Model	Accuracy	Precision	Recall	F1-score
0	SVM	0.954677	0.946590	0.958782	0.952647
1	Logistic Regression	0.942428	0.927546	0.953396	0.940293
2	Random Forest	0.933519	0.929591	0.930679	0.930135
3	RNN	0.952784	0.953966	0.946370	0.950153
4	LSTM	0.955568	0.969896	0.935597	0.952438
5	GRU	0.957572	0.962206	0.948009	0.955055
6	SVM + GRU	0.964031	0.964900	0.959251	0.962067
7	RF + GRU	0.963808	0.963790	0.959953	0.961868
8	RF + LSTM	0.961915	0.968735	0.950585	0.959574
9	SVM + LSTM	0.961136	0.968011	0.949649	0.958742
10	SVM + RNN	0.960245	0.958519	0.957845	0.958182
11	LR + GRU	0.960245	0.961983	0.954098	0.958025
12	RF + RNN	0.959243	0.956501	0.957845	0.957173

```

import pandas as pd
import matplotlib.pyplot as plt

# ----- Data -----
data = [
    ["SVM", 0.954677, 0.946590, 0.958782, 0.952647],
    ["Logistic Regression", 0.942428, 0.927546, 0.953396, 0.940293],
    ["Random Forest", 0.933519, 0.929591, 0.930679, 0.930135],
    ["RNN", 0.952784, 0.953966, 0.946370, 0.950153],
    ["LSTM", 0.955568, 0.969896, 0.935597, 0.952438],
    ["GRU", 0.957572, 0.962206, 0.948009, 0.955055],
    ["SVM + GRU", 0.964031, 0.964900, 0.959251, 0.962067],
    ["RF + GRU", 0.963808, 0.963790, 0.959953, 0.961868],

```

```
["RF + LSTM", 0.961915, 0.968735, 0.950585, 0.959574],
["SVM + LSTM", 0.961136, 0.968011, 0.949649, 0.958742],
["SVM + RNN", 0.960245, 0.958519, 0.957845, 0.958182],
["LR + GRU", 0.960245, 0.961983, 0.954098, 0.958025],
["RF + RNN", 0.959243, 0.956501, 0.957845, 0.957173],
]

columns = ["Model", "Accuracy", "Precision", "Recall", "F1-score"]
df = pd.DataFrame(data, columns=columns)

# ----- Accuracy -----
plt.figure(figsize=(10,5))
plt.plot(df["Model"], df["Accuracy"])
plt.xticks(rotation=45)
plt.title("Accuracy Comparison")
plt.xlabel("Models")
plt.ylabel("Accuracy")
plt.show()

# ----- Precision -----
plt.figure(figsize=(10,5))
plt.plot(df["Model"], df["Precision"])
plt.xticks(rotation=45)
plt.title("Precision Comparison")
plt.xlabel("Models")
plt.ylabel("Precision")
plt.show()

# ----- Recall -----
plt.figure(figsize=(10,5))
plt.plot(df["Model"], df["Recall"])
plt.xticks(rotation=45)
plt.title("Recall Comparison")
plt.xlabel("Models")
plt.ylabel("Recall")
plt.show()

# ----- F1-score -----
plt.figure(figsize=(10,5))
```

```
plt.plot(df["Model"], df["F1-score"])
plt.xticks(rotation=45)
plt.title("F1-score Comparison")
plt.xlabel("Models")
plt.ylabel("F1-score")
plt.show()
```